

ACX Platform (HMS): Hospitality Management System — Troubleshooting Guide

Inspironics Corporation

Program Documentation, ACX Platform HMS • Baseline Issue (API v0.3.0) • September 25, 2026

Abstract—This guide supports diagnosis of incidents on ACX HMS, the Inspironics hospitality management platform. It adopts a decision-tree method that isolates a report to one of six domains — deployment, identity, authorization, stay and room workflow, services and devices, and dashboard and reporting — before any deep investigation begins, and it separates genuine faults from designed behaviour that is frequently mistaken for one. It documents the evidence sources available for each domain, the error taxonomy and its mapping to the messages users actually see, the capabilities the baseline leaves unconnected, and the information an escalation must carry. Its governing observation is that in a platform whose refusals are deliberate, most reports are not faults.

Index Terms—fault diagnosis, error codes, authorization, stay lifecycle, room status, database connectivity, incident escalation, configuration.

I. INTRODUCTION

Support documentation succeeds when it shortens the time from symptom to cause. This guide complements the User Manual [1] and the Quick Reference [2].

ACX HMS differs from most property systems a support team will have handled in one respect that dominates triage: it is designed to refuse. It refuses sign-in to accounts that belong to another platform. It hides modules a role may not read and refuses their data on the server. It refuses to check a guest in to a stay with no room, to release a room a live stay still holds, and to cancel a stay that is already in house. Each of those produces a user report that reads exactly like a fault.

The guide therefore orders its checks so that designed behaviour is eliminated first, at low cost, before anyone reads a log. Four checks precede any investigation:

1. Query `GET /health` and `GET /api/v1/health/db`. A failure here makes every downstream symptom a consequence rather than a cause. The second reports the database, schema, server version and Alembic revision.
2. Read the `error.code` and `error.message` in the response. Every HMS failure carries one of nine codes (§VII); the code alone usually names the domain.
3. Call `GET /auth/me` as the reporting user. It returns the user's platform, role types, roles, facilities and the complete module permission set — the answer to most “I cannot see it” reports.
4. Check the capability is connected in this baseline (§VI). An unconnected feature explains a disabled button or an informational toast without any fault.

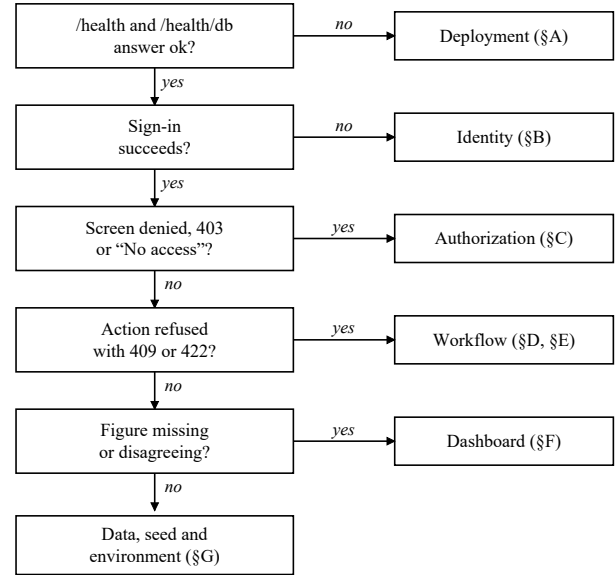


Fig. 1. Diagnostic decision tree. Designed refusals are eliminated before logs are read.

II. DIAGNOSTIC METHODOLOGY

The triage sequence is a binary decision tree (Fig. 1). Each question is cheap to answer — a browser tab, a response body or one authenticated call — and eliminates a whole class of causes. Swagger UI at `/docs` reproduces any request with the user's token, which removes the client from the question.

III. EVIDENCE SOURCES

HMS records enough in its own tables that a diagnosis can rest on recorded state rather than on reproduction. Table I maps each fault domain to the source that should be consulted first.

TABLE I
FAULT DOMAIN TO FIRST EVIDENCE SOURCE

Domain	First place to look
Any request	Response <code>error.code</code> , <code>message</code> , <code>detail</code> ; HTTP method and path
Deployment	<code>/health/db</code> fields <code>status</code> , <code>server_version</code> , <code>alembic_revision</code> , <code>latency_ms</code> ; Unicorn console
Identity	<code>app_user.user_name</code> and <code>password_hash</code> ; the user's <code>user_role</code> rows and <code>role_type</code>
Authorization	GET <code>/auth/me</code> permissions; the role matrix at GET <code>/roles/{id}/permissions</code>
Stays and rooms	GET <code>/stays/{id}</code> status and actual times; <code>/stays/{id}/room-allocations</code> ; GET <code>/occupancy/{room}</code>
Services, devices	request status and <code>status_reason</code> ; <code>incident_history</code> rows; device <code>config_status</code>
Dashboard	the tile's own endpoint; <code>python -m scripts.verify_dashboard_kpis</code>
Data and seed	<code>python -m seeds.run_seed --report</code> ; the test suite on a fresh seed

The API logs through the `hms.api` and `hms.auth` loggers. Logs deliberately contain no passwords and no tokens — so a log will confirm a failed sign-in for a username without disclosing what was typed, which is correct and is not a gap to be closed. Records carry `created_by` and `modified_by`, check-out records `checkout_initiated_by`, and every incident transition appends a history row; these are the record of who did what.

IV. FAULT DOMAINS

A. Deployment and Startup

503 `service_unavailable` — *Database is unavailable*. — means the API is up and PostgreSQL is not reachable with the configured credentials. The most common cause is a missing `backend/.env`: it is not committed, so a fresh clone falls back to the code defaults (password `postgres`, development `secret`) and fails to connect wherever the local password differs. Check `POSTGRES_HOST`, `POSTGRES_PORT` and `POSTGRES_PASSWORD`, or a `DATABASE_URL` that overrides them all.

A healthy `/health/db` does not prove the *right* database. Machines with two PostgreSQL servers (for example 18 on 5432 and 17 on 5433) will serve a valid, correctly migrated and entirely different database; compare `server_version` with the server that holds the data. An empty `alembic_revision` means `alembic` upgrade head was never run.

In the client, *Cannot reach the HMS API. Is the backend running?* is a network-level failure: the API is down, or the Vite proxy target (`VITE_API_PROXY_TARGET`, default `http://127.0.0.1:8000`) does not match where Unicorn listens. Outside development the service refuses to sign tokens with the default `JWT_SECRET_KEY`, and a browser on another origin needs that origin in `CORS_ORIGINS`.

B. Identity and Sign-In

401 *Invalid username or password*. has five usual causes, in order of likelihood:

- **No seeded account can sign in.** Every seeded password hash is a placeholder rejected before `bcrypt` runs; an administrator must set a real hash first. Accounts provisioned by hand on one machine are data, not source, and do not exist in a fresh clone.
- **E-mail entered instead of username.** Only `user_name` is matched; the e-mail column is never consulted.
- **Case or whitespace.** The comparison is exact and untrimmed; a trailing space from a paste fails.
- **Wrong database (§A)**, which holds the schema but not the user.
- **Password set outside the 8–72 character rule** through a direct database edit.

403 *This account belongs to the {platform} application and cannot sign in to HMS Web*. means the credentials were right and the role is not `admin` or `manager`. Staff, guest and system accounts are refused by design. An account with no role at all — for example one created without `role_ids`, or on a database that was migrated but not seeded — lands here too.

Your session has expired. Please sign in again. follows any 401 after sign-in. Tokens last 60 minutes by default and there is no refresh token. Restarting the API with a changed `JWT_SECRET_KEY` invalidates every issued token at once; sign-in itself still works. A brief *Restoring your session...* followed by a return to login is a stored token (`hms.access_token`) that no longer validates — the client clears it and recovers.

C. Authorization and Visibility

“The page is missing” is the single most common report, and it is almost always a matter of role rather than a fault. The server's 403 names the module and the access level — *Role does not grant write access to the 'bookings' module*. — and the client's *Access denied* page names the module whose read is missing. Compare with permissions from `/auth/me`; a matrix change takes effect on the user's next request without signing out.

Three combinations produce a visible screen with refused data, and all are correct behaviour:

- **Tickets.** The route needs `tickets`; its data needs `service_tracking`. Grant both or neither.
- **Duty Manager building and floor pickers.** The role holds no `facility_management` grant, so building, floor and room lists are refused on Dashboard and Occupancy.
- **Room environment tab.** Power, energy, temperature and air quality each need `caleido_network` or `reports`.

D. Stays, Rooms and Occupancy

A refused stay action is the platform functioning. The codes distinguish why:

- 422 *Allocate a room before checking this stay in.* — allocate first.
- 409 *This stay is already checked in. / A stay with status '{s}' cannot be checked in.* — the stay is active, checked out or cancelled.
- 409 *This stay has not been checked in.* — check-out attempted on a pending stay; cancel it instead.

- 409 *A checked-in stay cannot be cancelled; check it out instead.*
- 409 *That room is already allocated to stay {ref}. — the room is held by another live stay.*
- 409 *Stay {ref} still holds this room; check it out first. — a manual room status change was attempted on an Allotted or Occupied room.*
- 422 *Use the check_out action... / Use the cancel_stay action... — terminal states cannot be set directly.*

After check-out the room is Available but keeps any housekeeping conditions it had; a room that “still shows Need Maintenance” after check-out is correct until housekeeping clears it. The **Overdue** tag appears when the expected check-out has passed and the stay is still in house; extending the stay removes it.

The occupancy donut counts room status; *Rooms in house* counts rooms held by in-house stays. They are two sources of truth the baseline does not reconcile, so a room set Occupied without a stay, or seed data that predates the stay rules, makes them differ. Record such a difference as data drift, not as a dashboard fault.

E. Services, Planning, Job Orders and Devices

A completed service request cannot be cancelled. and *A completed planned service cannot be cancelled.* are 422 by design. Service requests also refuse a category that does not belong to the request's service type. Planned services refuse an end date before the start date, assignees who are not staff, and a payload with neither rooms nor non-room comments. A removed (soft-deleted) planned service cannot be edited. Job orders refuse a duplicate reference (409) and a completion date before commencement.

Devices return 409 *This device is already '{status}'.* for a repeated commission, decommission or maintenance action. Limit configurations refuse duplicates (409) and a low threshold that is not below the high threshold. Leaving the Resolved state on an incident records a reopening in its history; that entry is expected.

F. Dashboard and Reports

Each dashboard tile calls its own endpoint and reports its own failure. *No access* is a 403 for that tile's module, *Session expired* a 401, and *Unavailable* a 5xx or network failure; a tile never shows zero in place of an error. `python -m scripts.verify_dashboard_kpis` recomputes the tiles against the database for a named user.

An unknown report key returns 404 *No report named '{key}'.* followed by the valid keys. A 422 *A query value is not valid for its column.* on a filtered list means a filter value outside the column's enumeration — typically a status name spelled differently from the vocabulary in the Quick Reference.

G. Data, Seed and Environment

The backend test suite (about 1 195 tests) runs against a live database and asserts the seeded baseline, so it fails on a database that has been used interactively. Re-seed a fresh database before treating a test failure as a regression. Build or

install failures from Node or Python version drift do not present as 401 or 403; a clean HTTP error is never a tooling problem.

V. DESIGNED BEHAVIOUR MISTAKEN FOR FAULTS

Table II lists the behaviours most often reported as defects. Confirming one of these closes an incident; none of them requires escalation.

TABLE II
EXPECTED BEHAVIOURS AND THEIR REASONS

Observation	Reason
Staff member cannot sign in to HMS Web	Staff roles belong to the mobile platform
Seeded user cannot sign in	Seeded hashes are placeholders by design
Sidebar entry absent for one user	Role lacks read on that module
Tickets screen opens, list refused	Data is governed by <code>service_tracking</code>
Duty Manager sees no buildings	No <code>facility_management</code> grant
Check-in refused	No room allocated, or stay not pending
Room cannot be set Available	A live stay still holds it
Checked-in booking cannot be cancelled	Check-out is the only exit from active
Room still Need Maintenance after check-out	Conditions are independent of status
Donut and Rooms in house differ	Two unreconciled sources of occupancy truth
Invoice button disabled	No tariff or tax data; not connected
Signed out after an hour	Token lifetime 60 minutes; no refresh

VI. UNCONNECTED CAPABILITIES

Before diagnosing a fault, establish whether the capability exists in this baseline. The following are present as data, screens or buttons but are deliberately not connected: invoice generation and payments; bulk booking upload (*Bulk upload is not connected yet*); default key management (*Default key management is not connected yet*, and its Reset button has no action); PDF export (reports export to Excel only); delivery of notifications by e-mail, SMS or push (notification read state is local to the browser); scheduler jobs and device commands; and firmware management, whose module exists in the matrix but has no screen. The retired Power, Energy and Room View routes redirect to Occupancy. A report against any of these is a change request, not an incident.

VII. ERROR TAXONOMY

Every failure returns `{"error": {code, message, detail}}`. Table III maps each code to the message the client shows. Incident records should carry the code and the server message rather than a paraphrase.

TABLE III
ERROR CODES AND CLIENT MESSAGES

HTTP	Code	Client shows
0	(network)	Cannot reach the HMS API. Is the backend running?
400	bad_request	server message
401	unauthorized	Your session has expired. Please sign in again. (login: Invalid username or password)
403	forbidden	Your role does not grant access to this module. (login: platform message)
404	not_found	That record no longer exists.
409	conflict	server message, else That change conflicts with existing data.
422	validation_error	field: message pairs from detail
500	internal_error	The HMS API failed to handle this request.
503	service_unavailable	The HMS database is unavailable. Please try again shortly.

A failed change shows the toast *Could not save* with the message. Reads retry twice, and only on 5xx or network failures; a 4xx is never retried, because repeating it cannot change the answer.

VIII. ESCALATION

An escalation should carry, at minimum: the HTTP status, `error.code` and exact message; the method, path and time of the request; the acting username with the roles and permissions from `/auth/me`; the stay reference, room number or record identifier involved; the `/health/db` output including `alembic_revision`; the Git commit deployed; and the relevant Uvicorn log lines around the time of the request.

Three prohibitions apply. Never include `JWT_SECRET_KEY`, database passwords, password hashes or bearer tokens in a ticket. Never paste guest identity documents or personal details into a support system — reference the stay by its `STY-` number. And never resolve an incident by granting a broader role, editing room or stay status directly in the database, or relaxing the platform gate: each of those bypasses the rules the server enforces and converts one incident into a data-integrity problem.

Escalate to engineering when a request returns 500 `internal_error`, when one room is held by two live stays, when a user obtains data the module matrix should refuse, or when a stay's status and its actual check-in and check-out times disagree. These four are integrity conditions rather than ordinary faults.

IX. ASSESSMENT AND LIMITATIONS

Strengths. The decision tree and the four first checks give a repeatable method that resolves most reports before any log is read. The single error envelope gives users, support and engineering one vocabulary, and `/auth/me` answers every visibility question from the server's own view.

Limitations of this baseline. Responses carry no request or correlation identifier, so log lines are matched by time and path. There is no append-only audit table: authorship rests on `created_by`, `modified_by` and history rows. There is no

continuous-integration pipeline and no frontend test suite, so a regression is established by the backend suite on a fresh seed. Latency, capacity and recovery objectives are not yet set. This guide will be reissued against the delivered build, at which point these sections can carry identifiers and thresholds rather than descriptions.

X. CONCLUSION

ACX HMS yields a diagnosis quickly when triage begins from the right question. The first question is not what broke; it is whether anything did. A platform that refuses other platforms' accounts, hides what a role may not read, and refuses stay actions whose preconditions are unmet will generate reports that resemble faults and are not. Eliminating those in four cheap checks, then following the error code to its domain, resolves the majority of incidents without reproduction — and leaves the remainder well enough documented to be worth an engineer's time.

REFERENCES

- [1] Inspironics Corporation, “INS-ACX-HMS-UM-1.0 — ACX HMS User Manual,” Sep. 2026.
- [2] Inspironics Corporation, “INS-ACX-HMS-QR-1.0 — ACX HMS Quick Reference,” Sep. 2026.
- [3] Inspironics Corporation, “HMS_WORKING_ENVIRONMENT_BASELINE — Reference Machine Audit,” 2026.
- [4] Inspironics Corporation, “PHASE2_4_AUTHENTICATION — JWT Authentication and Module Permissions,” backend/docs, 2026.
- [5] Inspironics Corporation, “PHASE11_DASHBOARD_INTEGRATION — Dashboard Wiring and Its Gaps,” backend/docs, 2026.
- [6] Inspironics Corporation, “FINAL_HMS_DATABASE_BLUEPRINT — HMS Database Blueprint of Record,” backend/docs, 2026.